STUDENT NAME: AMANDEEP	**MODULE: My Practice**	**CODE: SPRING_007_MSERV**
BATCH: ABES_JAS26_FSD JAVA_IIIYear_Batch 02	**COURSE:** Working with Spring Framework with AI	**STATUS:** In Progress

Practice Worksheet – Spring Boot Microservices – Service Discovery & Routing

Architect decoupled microservice ecosystems with Eureka Service Discovery, Spring Cloud Gateway routing, and declarative OpenFeign REST clients.

KEY LEARNING OBJECTIVES:

- Design and configure microservice architectures and inter-service communication.
- Adhere to layered enterprise architecture (Controller -> Service -> Repository).
- Enforce database integrity, transaction boundaries, and defensive validation.
- Verify correctness with Spring Boot slice tests and MockMvc integration testing.

STEP-BY-STEP WORKSHEET INSTRUCTIONS:

Step 1: Analyze enterprise specifications and configure Spring dependencies.
Step 2: Implement domain models, repositories, and transactional services.
Step 3: Expose clean endpoints or beans with proper lifecycle management.
Step 4: Document test curl requests, JUnit tests, and verification logs.

EXERCISE 7.1: SPRING ARCHITECTURE & BEAN CONFIGURATION (EUREKA SERVER & CLIENT SETUP)

Problem Statement: Define the core Spring configuration or component setup for Spring Boot Microservices – Service Discovery & Routing. Establish proper Spring stereotypes, dependency injection wiring, and configuration properties.

Reference Baseline / Scenario / Code:

```
@Configuration
public class Sping_Boot_MicroservicesConfig { // Configuration for Spring Boot Microservices – Service Discovery & Routing
}
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] Spring Stereotype Annotations: _____

[2] Bean Definition / Interface Wiring: _____

[3] Property Configuration Keys: _____

EXERCISE 7.2: SERVICE & REPOSITORY LAYER IMPLEMENTATION (API GATEWAY ROUTE DEFINITIONS)

Problem Statement: Implement the transactional service and repository logic for Spring Boot Microservices – Service Discovery & Routing. Write the business rules, data access queries, or event publishing methods.

Reference Baseline / Scenario / Code:

```
@Service
public class Sping_Boot_MicroservicesService { // Service logic for Spring Boot Microservices – Service Discovery & Routing
}
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] Transactional Service Methods: _____

[2] Repository Query / Operations: _____

[3] Business Logic Implementation: _____

EXERCISE 7.3: VALIDATION, EXCEPTION HANDLING & ERROR BOUNDARIES (OPENFEIGN CLIENT INTERFACE)

Problem Statement: Implement validation rules and error-handling mechanisms for Spring Boot Microservices – Service Discovery & Routing. Handle resource not found, invalid inputs, or database constraint violations.

Reference Baseline / Scenario / Code:

```
@ExceptionHandler(ResourceNotFoundException.class)
public ResponseEntity<ErrorResponse> handleNotFound(...) { ... }
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] Jakarta Validation Rules: _____

[2] Custom Exception Class: _____

[3] @RestControllerAdvice Mapping: _____

EXERCISE 7.4: TESTING & VERIFICATION EXECUTION (RESILIENCE4J CIRCUIT BREAKER)

Problem Statement: Construct a Spring Boot test (e.g. @WebMvcTest, @DataJpaTest, or @SpringBootTest) verifying that Spring Boot Microservices – Service Discovery & Routing functions deterministically.

Reference Baseline / Scenario / Code:

```
@SpringBootTest
class {fname.split('.')[0]}Test {{ @Test void testFunctionality() {{ ... }} }}
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] Spring Boot Test Annotations: _____

[2] MockMvc / Repository Assertions: _____

[3] Verification Execution Log: _____

SELF-REVIEW & VERIFICATION CHECKLIST:

- [ ] **IoC Best Practices:** Beans injected via constructor injection; proper scopes applied.
- [ ] **Data Layer Security:** Queries parameterized; zero raw SQL concatenation.
- [ ] **Transaction Safety:** @Transactional boundaries applied to mutation operations.
- [ ] **REST Conventions:** HTTP status codes and error responses follow standard conventions.
- [ ] **Test Isolation:** Slice tests mock dependencies cleanly without flaky state.

CRITICAL REFLECTION & LEARNING SYNTHESIS:

- *Why is dependency injection via constructor preferred over field injection in Spring Boot Microservices – Service Discovery & Routing?*
- *How do transaction boundaries (@Transactional) safeguard data consistency during multi-step operations?*

Reflection Notes:

_____

_____

COMPLETION STATUS:

[ ] COMPLETED (Meets All Solution Criteria)

[ ] NEEDS REVISION (Resubmit Corrected Work)

Evaluator Signature: _____

EVALUATOR & MENTOR REMARKS:

Feedback Notes:

_____

Rubric Evaluation: [ ] Correctness [ ] Code Quality [ ] Edge Cases